RELATORIO TECNICO

Sistema de cadastro de locais urbanos

Disciplina: Estrutura de Dados - ADS, 3o semestre
Linguagem: C11 | Tema B: Locais urbanos
Estruturas: BST e fila dinamica | Estudante: Diego Gallo
Data: 8 de novembro de 2026

1. Objetivo e modelagem

O programa de terminal cadastra locais urbanos em uma BST ordenada pelo ID. Cada registro guarda ID inteiro, nome, categoria e bairro. Campos textuais usam char[80], com ate 79 caracteres mais o terminador nulo. Espacos externos sao removidos e textos vazios rejeitados.

Um no da BST contem o registro e ponteiros para os filhos esquerdo e direito. IDs menores ficam a esquerda, maiores a direita e duplicados sao rejeitados. A estrutura linear escolhida e uma fila encadeada independente. Cada no armazena um ID positivo e um ponteiro para o proximo. A fila permite IDs repetidos e nao exige cadastro correspondente na BST.

O menu oferece cadastro, busca, remocao e percurso em ordem da BST; insercao, remocao e exibicao da fila; e encerramento. O encerramento libera os nos alocados nas duas estruturas.

2. Organizacao

Os cabecalhos estao em include/ e as implementacoes em src/. bst.c concentra as operacoes da arvore; fila.c implementa a fila; entrada.c valida dados; main.c integra o menu.

3. Implementacao

A BST implementa insercao, busca, remocao e percurso em ordem. A remocao trata no sem filhos, com um filho e com dois filhos; no ultimo caso, substitui pelo sucessor em ordem e libera o no sucessor. O percurso em ordem apresenta IDs em ordem crescente.

A fila insere no final e remove do inicio, com ponteiros para inicio e fim. Ao remover o ultimo elemento, ambos voltam a nulo. A estrutura possui exibicao e rotina de liberacao de memoria.

entrada.c le linhas com fgets e converte inteiros com strtol, verificando limites e erros. IDs aceitos sao positivos e cabem em int; o menu aceita opcoes de 1 a 8. Textos vazios, linhas longas demais e valores nao numericos sao rejeitados.

4. Testes e evidencias

O workflow do GitHub Actions compila aplicativo e testes com GCC e as opcoes C11, -Wall, -Wextra e -pedantic. No commit 7c99adf, o workflow terminou com sucesso.

A suite automatizada verificou insercao e busca; ID duplicado e zero; ordem crescente; remocao de folha, no com um filho e no com dois filhos; IDs ausentes; comportamento FIFO, IDs repetidos e fila vazia.

O teste de integracao cadastrou 40, 20 e 60 no menu e conferiu a listagem 20, 40, 60. Entradas textuais invalidas, limite de 79 caracteres e encerramento com dados estao descritos no roteiro, mas nao foram registrados como executados nesta evidencia automatizada.

5. Analise de complexidade

Seja n o numero de registros da BST, h a altura, q o tamanho da fila e L o tamanho do texto.

Operacao Complexidade
Busca, insercao e remocao na BST O(h)
Caso medio, arvore aproximadamente equilibrada O(log n)
BST degenerada, pior caso O(n)
Listagem em ordem O(n)
Enfileirar e desenfileirar O(1)
Exibir fila O(q)
Validar ou copiar texto O(L)
Memoria das estruturas O(n + q)

A BST nao e auto-balanceada. A ordem de insercao pode gerar uma arvore degenerada; nesse caso, as operacoes da BST podem chegar a O(n).

6. Conclusao e ferramentas utilizadas

A implementacao aplica ponteiros, alocacao dinamica, modularizacao e as estruturas solicitadas. Os testes automatizados confirmaram os casos principais da BST, da fila e da listagem ordenada pelo menu. A principal limitacao e a ausencia de balanceamento automatico.

Utilizei uma ferramenta de IA como apoio para organizar o repositorio, redigir e revisar o codigo, o roteiro de testes e este relatorio. O uso foi registrado no repositorio; o codigo deve
ser explicado individualmente durante a demonstracao da disciplina.